

Project — Class 3

GroupGo

Mobile Application Development

Student application:	GroupGo
Complexity level:	Medium
Task:	Class 3 — Data model & API contracts

1. Data Specification

The table below lists every piece of data GroupGo needs to persist or transfer, together with its storage location, lifetime, and reason for storage.

Data	Location	Lifetime	Reason
Transportation options (routes, time, cost)	API (Google Maps / transit API)	Session	Compare options on demand
Current GPS coordinates	Memory (ViewModel)	Session	Show on map, use as start point
Travel groups (name, members)	Room DB	Persistent	Users need groups across sessions
Group members	Room DB	Persistent	Needed for expense splitting
Trip history (route, cost, date, transport method)	Room DB	Persistent	History screen — past trips per group
Expense balances (who paid, who owes what)	Room DB	Persistent	Balance screen — track debts over time
User settings (monthly salary, working hours)	DataStore	Persistent	Time-Value Calculator preferences
Active UI state (loading, error, selected priority)	ViewModel	Screen lifetime	Survive rotation, drive UI

2. Data Classes

2a. Domain Models (domain/model/)

Clean Kotlin objects used across the app — no framework annotations, no JSON keys.

```
// domain/model/TransportOption.kt
data class TransportOption(
    val mode: String,           // e.g. "bus", "car", "walk"
    val durationMinutes: Int,
    val costPln: Double,
    val distanceKm: Double,
    val timeValueScore: Double // computed from user's salary settings
)
```

Listing 1: TransportOption.kt — domain model

```
// domain/model/TravelGroup.kt
data class TravelGroup(
    val id: String,
    val name: String,
    val memberNames: List<String>,
    val createdAtEpoch: Long
)
```

Listing 2: TravelGroup.kt — domain model

```
// domain/model/TripRecord.kt
data class TripRecord(
    val id: String,
    val groupId: String,
    val origin: String,
    val destination: String,
    val transportMode: String,
    val totalCostPln: Double,
    val tripDateEpoch: Long // Unix timestamp of when trip was made
)
```

Listing 3: TripRecord.kt — domain model

```
// domain/model/ExpenseBalance.kt
data class ExpenseBalance(
    val groupId: String,
    val memberName: String,
    val amountOwedPln: Double, // positive = owes money
    val amountPaidPln: Double
)
```

Listing 4: ExpenseBalance.kt — domain model

```
// domain/model/UserSettings.kt
data class UserSettings(
    val monthlySalaryPln: Double,
    val avgWorkingHoursPerMonth: Double
)
```

Listing 5: UserSettings.kt — domain model

2b. DTOs (API Responses) (data/remote/dto/)

Mirror the JSON structure received from the OpenRouteService API.

```
// data/remote/dto/RouteResponseDto.kt
data class RouteResponseDto(
    @SerializedName("routes") val routes: List<RouteDto>
)

data class RouteDto(
    @SerializedName("summary") val summary: RouteSummaryDto,
    @SerializedName("segments") val segments: List<SegmentDto>
)

data class RouteSummaryDto(
    @SerializedName("distance") val distanceMeters: Double,
    @SerializedName("duration") val durationSeconds: Double
)

data class SegmentDto(
```

```

        @SerializedName("distance") val distanceMeters: Double,
        @SerializedName("duration") val durationSeconds: Double
    )

```

Listing 6: RouteResponseDto.kt — mirrors OpenRouteService /directions JSON

2c. Room Entities (data/local/entity/)

```

// data/local/entity/TravelGroupEntity.kt
@Entity(tableName = "travel_groups")
data class TravelGroupEntity(
    @PrimaryKey val id: String,
    val name: String,
    val memberNames: String,          // stored as JSON string
    val createdAtEpoch: Long
)

```

Listing 7: TravelGroupEntity.kt — travel groups

```

// data/local/entity/TripRecordEntity.kt
@Entity(tableName = "trip_records")
data class TripRecordEntity(
    @PrimaryKey val id: String,
    val groupId: String,
    val origin: String,
    val destination: String,
    val transportMode: String,
    val totalCostPln: Double,
    val tripDateEpoch: Long
)

```

Listing 8: TripRecordEntity.kt — trip history

```

// data/local/entity/ExpenseBalanceEntity.kt
@Entity(tableName = "expense_balances",
    primaryKeys = ["groupId", "memberName"])
data class ExpenseBalanceEntity(
    val groupId: String,
    val memberName: String,
    val amountOwedPln: Double,
    val amountPaidPln: Double
)

```

Listing 9: ExpenseBalanceEntity.kt — expense balances

3. API Definition

3a. Retrofit Service Interface (data/remote/)

```

// data/remote/RouteApiService.kt
interface RouteApiService {

    @GET("v2/directions/{profile}")
    suspend fun getRoute(
        @Path("profile")          profile: String,    // "driving-car", "foot-walking",
                                                // "cycling-regular"
        @Query("start")           start: String,      // "longitude,latitude"
        @Query("end")             end: String,        // "longitude,latitude"
        @Header("Authorization")   apiKey: String = BuildConfig.ORS_API_KEY
    ): Response<RouteResponseDto>
}

```

Listing 10: RouteApiService.kt

3b. Room DAO Interfaces (data/local/dao/)

```
// data/local/dao/TravelGroupDao.kt
@Dao
interface TravelGroupDao {

    @Query("SELECT * FROM travel_groups ORDER BY createdAtEpoch DESC")
    fun getAllGroups(): Flow<List<TravelGroupEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertGroup(entity: TravelGroupEntity)

    @Delete
    suspend fun deleteGroup(entity: TravelGroupEntity)

    @Query("SELECT * FROM travel_groups WHERE id = :groupId")
    suspend fun getGroupById(groupId: String): TravelGroupEntity?

}
```

Listing 11: TravelGroupDao.kt

```
// data/local/dao/TripRecordDao.kt
@Dao
interface TripRecordDao {

    @Query("SELECT * FROM trip_records ORDER BY tripDateEpoch DESC")
    fun getAllTrips(): Flow<List<TripRecordEntity>>

    @Query("SELECT * FROM trip_records WHERE groupId = :groupId " +
        "ORDER BY tripDateEpoch DESC")
    fun getTripsByGroup(groupId: String): Flow<List<TripRecordEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTrip(entity: TripRecordEntity)

    @Delete
    suspend fun deleteTrip(entity: TripRecordEntity)

}
```

Listing 12: TripRecordDao.kt

```
// data/local/dao/ExpenseBalanceDao.kt
@Dao
interface ExpenseBalanceDao {

    @Query("SELECT * FROM expense_balances WHERE groupId = :groupId")
    fun getBalancesForGroup(groupId: String): Flow<List<ExpenseBalanceEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertBalance(entity: ExpenseBalanceEntity)

    @Query("DELETE FROM expense_balances WHERE groupId = :groupId")
    suspend fun clearBalancesForGroup(groupId: String)

}
```

Listing 13: ExpenseBalanceDao.kt

3c. Repository Interfaces (domain/repository/)

```
// domain/repository/RouteRepository.kt
interface RouteRepository {
    suspend fun getRouteOptions(
        originLat: Double,
        originLon: Double,
        destLat: Double,
        destLon: Double
    ): Result<List<TransportOption>> // returns all modes compared
}
```

Listing 14: RouteRepository.kt — domain contract

```
// domain/repository/GroupRepository.kt
interface GroupRepository {
    fun getAllGroups(): Flow<List<TravelGroup>>
    suspend fun createGroup(group: TravelGroup)
    suspend fun deleteGroup(groupId: String)
}
```

Listing 15: GroupRepository.kt — domain contract

```
// domain/repository/TripRepository.kt
interface TripRepository {
    fun getAllTrips(): Flow<List<TripRecord>>
    fun getTripsByGroup(groupId: String): Flow<List<TripRecord>>
    suspend fun saveTrip(trip: TripRecord)
    suspend fun deleteTrip(tripId: String)
}
```

Listing 16: TripRepository.kt — domain contract

```
// domain/repository/ExpenseRepository.kt
interface ExpenseRepository {
    fun getBalancesForGroup(groupId: String): Flow<List<ExpenseBalance>>
    suspend fun updateBalance(groupId: String, memberName: String,
        amountPaid: Double)
    suspend fun clearGroupBalances(groupId: String)
}
```

Listing 17: ExpenseRepository.kt — domain contract

```
// domain/repository/SettingsRepository.kt
interface SettingsRepository {
    fun getSettings(): Flow<UserSettings> // from DataStore
    suspend fun saveSettings(settings: UserSettings)
}
```

Listing 18: SettingsRepository.kt — domain contract

4. Application Skeleton Structure

The project follows Clean Architecture with three top-level layers. Every package contains at least a stub file so the project compiles.

```
com.example.groupgo/
├──
├── di/                                # Hilt modules
├── AppModule.kt                      # provides Retrofit, Room, OkHttp
├── RepositoryModule.kt               # binds interfaces to implementations
└──
```

```

domain/                                # pure Kotlin – no Android dependencies
  model/
    TransportOption.kt
    TravelGroup.kt
    TripRecord.kt
    ExpenseBalance.kt
    UserSettings.kt
  repository/
    RouteRepository.kt
    GroupRepository.kt
    TripRepository.kt
    ExpenseRepository.kt
    SettingsRepository.kt
  data/                                # framework-dependent implementations
    remote/
      dto/
        RouteResponseDto.kt
      mapper/
        RouteMapper.kt
      RouteApiService.kt
    local/
      entity/
        TravelGroupEntity.kt
        TripRecordEntity.kt
        ExpenseBalanceEntity.kt
      dao/
        TravelGroupDao.kt
        TripRecordDao.kt
        ExpenseBalanceDao.kt
      AppDatabase.kt
      repository/
        RouteRepositoryImpl.kt
        GroupRepositoryImpl.kt
        TripRepositoryImpl.kt
        ExpenseRepositoryImpl.kt
        SettingsRepositoryImpl.kt
  ui/                                  # Jetpack Compose screens + ViewModels
    home/
      HomeScreen.kt
      HomeViewModel.kt
      HomeUiState.kt
    groups/
      GroupsScreen.kt
      GroupsViewModel.kt
      GroupsUiState.kt
    history/
      HistoryScreen.kt
      HistoryViewModel.kt
      HistoryUiState.kt
    balance/
      BalanceScreen.kt
      BalanceViewModel.kt
      BalanceUiState.kt
    settings/
      SettingsScreen.kt
      SettingsViewModel.kt
      SettingsUiState.kt

```

```

    navigation/
    AppNavGraph.kt

```

Listing 19: Full package structure (com.example.groupgo)

Package Checklist

Package	Status
com.example.groupgo.domain.model	✓
com.example.groupgo.domain.repository	✓
com.example.groupgo.data.remote.dto	✓
com.example.groupgo.data.remote.mapper	✓
com.example.groupgo.data.local.entity	✓
com.example.groupgo.data.local.dao	✓
com.example.groupgo.data.repository	✓
com.example.groupgo.ui.home	✓
com.example.groupgo.ui.groups	✓
com.example.groupgo.ui.history	✓
com.example.groupgo.ui.balance	✓
com.example.groupgo.ui.settings	✓
com.example.groupgo.ui.navigation	✓
com.example.groupgo.di	✓

Key Gradle Dependencies (build.gradle.kts — app module)

```

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    id("com.google.devtools.ksp")
    id("com.google.dagger.hilt.android")
}

android {
    namespace = "com.example.groupgo"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.groupgo"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
        buildConfigField("String", "ORS_API_KEY", "\"YOUR_API_KEY_HERE\"")
    }

    buildFeatures {

```

```

        compose = true
        buildConfig = true           // exposes BuildConfig.ORS_API_KEY
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {

    // Jetpack Compose BOM
    implementation(platform("androidx.compose:compose-bom:2024.09.00"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.activity:activity-compose:1.9.2")

    // Navigation Compose
    implementation("androidx.navigation:navigation-compose:2.8.0")

    // ViewModel + Lifecycle
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.4")

    // Room
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")

    // Retrofit + Gson
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

    // Hilt
    implementation("com.google.dagger:hilt-android:2.51.1")
    ksp("com.google.dagger:hilt-compiler:2.51.1")
    implementation("androidx.hilt:hilt-navigation-compose:1.2.0")

    // DataStore (for UserSettings)
    implementation("androidx.datastore:datastore-preferences:1.1.1")

    // Google Maps Compose
    implementation("com.google.maps.android:maps-compose:4.4.1")
    implementation("com.google.android.gms:play-services-location:21.3.0")

    // Core
    implementation("androidx.core:core-ktx:1.13.1")

    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.2.1")
}

```

```
}
```

Listing 20: build.gradle.kts (app module — full)